

Université Hassan II de
Casablanca
Faculté des Sciences & Techniques
Mohammedia

Licence Informatique, Réseaux & Multimédia

Collecte de Données Multimodales

Réalité Audio

Guide Complet des Mécanismes d'Acquisition des Sons

Réalisé par :
Prof. Abdellah ADIB

May 1, 2026

Contents

1	Introduction	2
1.1	Contexte et Objectifs	2
1.2	La Realite Audio	2
1.3	Vue d'Ensemble des Mecanismes	2
2	Microphone	4
2.1	Principe Fondamental	4
2.1.1	Architecture de la Capture	4
2.1.2	Parametres Fondamentaux	4
2.2	Capture avec PyAudio	5
2.2.1	Liste des Peripheriques Audio	6
2.3	Capture avec SoundDevice	7
2.3.1	Enregistrement en Boucle avec Callback	8
2.4	Comparaison des Bibliotheques	9
2.5	Bonnes Pratiques	9
3	Carte Son et Fichiers Audio	10
3.1	Principe Fondamental	10
3.1.1	Architecture Carte Son	10
3.1.2	Formats Audio Supportes	10
3.2	Lecture avec SciPy	11
3.2.1	Ecriture d'un Fichier WAV	12
3.3	Lecture avec SoundFile	12
3.4	Reechantillonnage et Normalisation	13
3.4.1	Detection d'Activite Vocale (VAD)	14
3.5	Bonnes Pratiques	16
4	Telechargement Audio	17
4.1	Principe Fondamental	17
4.2	Telechargement Simple avec Requests	17
4.2.1	Telechargement Multiple	18
4.3	Telechargement Asynchrone	19
4.4	Datasets Audio Publics	20
4.4.1	HuggingFace Datasets	20
4.4.2	Extraction depuis YouTube	22
4.5	Bonnes Pratiques	23
5	Streaming Audio en Temps Reel	24
5.1	Principe Fondamental	24
5.1.1	Architecture du Streaming	24
5.2	Streaming depuis le Microphone	24
5.3	Streaming HTTP	26

5.4	WebSocket pour l'Audio Temps Reel	27
5.5	Bonnes Pratiques	28
6	Generation Audio par IA	29
6.1	Principe Fondamental	29
6.1.1	Modeles Principaux	29
6.2	Synthese Vocale avec Coqui TTS	29
6.2.1	Parametres Avances	30
6.3	TTS via OpenAI API	31
6.4	Generation Musicale avec AudioCraft	32
6.5	Bonnes Pratiques	33
7	Extraction Audio depuis Video	34
7.1	Principe Fondamental	34
7.2	Extraction avec ffmpeg-python	34
7.2.1	Extraction par Intervalle Temporel	35
7.3	Extraction avec pydub	36
7.4	Separation de Sources avec Demucs	37
7.5	Transcription Automatique avec Whisper	38
7.6	Bonnes Pratiques	40
8	Conclusion	41
8.1	Tableau Comparatif des Mecanismes	41
8.2	Bonnes Pratiques Generales	41
8.3	Arbre de Decision	42
9	Bibliographie et Ressources	43
9.1	Ouvrages de Reference	43
9.2	Documentation Officielle	43
9.3	Tutoriels Recommandes	43
9.4	Jeux de Donnees pour Pratiquer	44
9.5	Aspects Legaux et Ethiques	44

Chapitre 1 Introduction

1.1 Contexte et Objectifs

La collecte de données constitue la première étape fondamentale de tout projet d'intelligence artificielle et d'apprentissage automatique. Sans données de qualité, même les algorithmes les plus sophistiqués ne peuvent produire de résultats pertinents. Ce rapport se concentre sur la **Realite Audio** de la collecte de données, c'est-à-dire l'ensemble des mécanismes permettant d'acquérir des sons et des signaux audio exploitables par un système informatique.

1.2 La Realite Audio

Le concept de **Realite Audio** désigne tous les procédés techniques permettant de capturer, générer ou obtenir des signaux sonores sous forme numérique. Contrairement à la simple lecture d'un fichier audio, la *Realite Audio* concerne l'acquisition systématique et programmable de sons dans un format exploitable (WAV, FLAC, MP3, OGG, Opus, etc.).

Les 6 mécanismes principaux de la Realite Audio sont :

1. **Microphone** : Capture programmatique du son ambiant en temps réel
2. **Carte Son / ADC** : Acquisition depuis une interface audio et fichiers locaux
3. **Telechargement Audio** : Obtention via HTTP/HTTPS, APIs et datasets publics
4. **Streaming Audio** : Capture en temps réel depuis des flux continus
5. **Generation IA** : Création synthétique avec TTS, clonage vocal, AudioCraft
6. **Extraction depuis Video** : Extraction de pistes audio depuis des fichiers video

1.3 Vue d'Ensemble des Mécanismes

Chaque mécanisme répond à des besoins spécifiques :

- **Microphone** : Dictée, reconnaissance vocale, surveillance acoustique, podcasting
- **Carte Son / ADC** : Instruments de musique, enregistrement studio, tests audio
- **Telechargement** : Datasets ML, agrégation de contenu audio, archives sonores
- **Streaming** : Radio en ligne, transcription live, analyse acoustique temps réel
- **Generation IA** : Création de contenu, synthèse vocale, augmentation de données
- **Extraction Video** : Sous-titrage automatique, analyse de discours, séparation de sources

Tous ces mecanismes convergent vers un objectif commun : obtenir des donnees audio sous forme de tableaux numpy (arrays de flottants) ou de fichiers audio standardises, independamment du mecanisme utilise.

Conseil

Choisissez le mecanisme en fonction de la **source** du son :

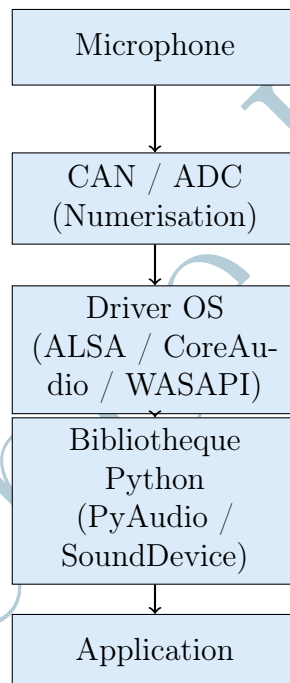
- Source = Voix / Ambiance ? → Microphone
- Source = Instrument / Mixage ? → Carte Son / ADC
- Source = Web / Dataset ? → Telechargement
- Source = Flux en direct ? → Streaming
- Source = Imaginee ? → Generation IA
- Source = Video ? → Extraction de piste audio

Chapitre 2 Microphone

2.1 Principe Fondamental

La capture audio via microphone permet d'enregistrer programmatiquement les sons ambiants et la parole en temps reel. Contrairement a un simple enregistrement manuel, la capture programmable permet l'automatisation, la detection d'activite vocale, et le traitement en temps reel du signal sonore.

2.1.1 Architecture de la Capture



2.1.2 Parametres Fondamentaux

Trois parametres definissent la qualite d'une capture audio :

- **Frequence d'echantillonnage** : Nombre d'echantillons par seconde (Hz). 8 kHz pour la telephonie, 16 kHz pour l'ASR, 44,1 kHz pour le CD, 48 kHz pour la video professionnelle.
- **Profondeur de bits** : Resolution de chaque echantillon. 16 bits (standard), 24 bits (studio), 32 bits flottants (traitement numerique interne).
- **Canaux** : 1 (mono) pour la parole et l'ASR, 2 (stereo) pour la musique.

2.2 Capture avec PyAudio

PyAudio est l'interface Python la plus complete pour PortAudio, le standard multi-plateformes.

Listing 2.1: Installation de PyAudio

```
pip install pyaudio
# Sur Ubuntu/Debian : sudo apt-get install portaudio19-dev
python3-pyaudio
```

Listing 2.2: Enregistrement simple avec PyAudio

```
import pyaudio
import wave
import datetime

def enregistrer_microphone(
    duree_secondes: int = 5,
    taux_echantillonnage: int = 16000,
    canaux: int = 1,
    dossier_sortie: str = "."
) -> str:
    """
    Enregistre le microphone pendant une duree donnee.

    Args:
        duree_secondes: Duree d'enregistrement en secondes
        taux_echantillonnage: Frequence d'echantillonnage en Hz
        canaux: Nombre de canaux (1 = mono, 2 = stereo)

    Returns:
        Chemin du fichier WAV cree
    """
    CHUNK = 1024
    FORMAT = pyaudio.paInt16

    timestamp =
        datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    nom_fichier = f"{dossier_sortie}/audio_{timestamp}.wav"

    pa = pyaudio.PyAudio()

    # Ouverture du flux d'entree
    stream = pa.open(
        format=FORMAT,
        channels=canaux,
        rate=taux_echantillonnage,
        input=True,
        frames_per_buffer=CHUNK
    )

    print(f"Enregistrement en cours ({duree_secondes}s)...")
```

```

frames = []

nb_chunks = int(taux_echantillonnage / CHUNK *
                duree_secondes)
for _ in range(nb_chunks):
    data = stream.read(CHUNK)
    frames.append(data)

# Liberation des ressources
stream.stop_stream()
stream.close()
pa.terminate()

# Sauvegarde au format WAV
with wave.open(nom_fichier, 'wb') as wf:
    wf.setnchannels(canaux)
    wf.setsampwidth(pa.get_sample_size(FORMAT))
    wf.setframerate(taux_echantillonnage)
    wf.writeframes(b''.join(frames))

print(f"Fichier sauvegarde : {nom_fichier}")
return nom_fichier

# Exemple d'utilisation
chemin = enregistrer_microphone(duree_secondes=5,
                                dossier_sortie="./enregistrements")

```

2.2.1 Liste des Peripheriques Audio

Listing 2.3: Enumeration des peripheriques audio disponibles

```

import pyaudio

def lister_peripheriques() -> list:
    """
    Liste tous les peripheriques audio disponibles.
    """
    pa = pyaudio.PyAudio()
    peripheriques = []

    print(f"Peripheriques audio detectes :
          {pa.get_device_count()}\n")

    for i in range(pa.get_device_count()):
        info = pa.get_device_info_by_index(i)
        if info['maxInputChannels'] > 0:
            peripheriques.append(info)
            print(f"[{i}] {info['name']}")
            print(f"      Canaux entree :
                  {info['maxInputChannels']}")

```



```

        print(f"        Taux par défaut :
              {int(info['defaultSampleRate'])} Hz")

    pa.terminate()
    return peripheriques

# Exemple
micros = lister_peripheriques()

```

2.3 Capture avec SoundDevice

SoundDevice est une alternative moderne a PyAudio, plus simple et basee sur NumPy.

Listing 2.4: Installation de SoundDevice

```
pip install sounddevice soundfile numpy
```

Listing 2.5: Enregistrement avec SoundDevice

```

import sounddevice as sd
import soundfile as sf
import numpy as np
import datetime

def enregistrer_sounddevice(
    duree_secondes: float = 5.0,
    taux_echantillonnage: int = 44100,
    canaux: int = 1,
    nom_fichier: str = None
) -> np.ndarray:
    """
    Enregistre depuis le microphone avec SoundDevice.

    Returns:
        Array numpy de l'audio enregistre (float32)
    """
    if nom_fichier is None:
        ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
        nom_fichier = f"audio_{ts}.wav"

    print(f"Enregistrement ({duree_secondes}s a
          {taux_echantillonnage} Hz)...")

    # Enregistrement bloquant
    audio = sd.rec(
        int(duree_secondes * taux_echantillonnage),
        samplerate=taux_echantillonnage,
        channels=canaux,
        dtype='float32'
    )
    sd.wait() # Attendre la fin de l'enregistrement

```

```

print("Enregistrement termine.")

# Sauvegarde
sf.write(nom_fichier, audio, taux_echantillonnage)
print(f"Fichier : {nom_fichier}")
print(f"Shape : {audio.shape}")
print(f"Duree : {len(audio) / taux_echantillonnage:.2f}s")

return audio

# Exemple
audio = enregistrer_sounddevice(duree_secondes=5.0,
                                taux_echantillonnage=16000)

```

2.3.1 Enregistrement en Boucle avec Callback

Listing 2.6: Capture en temps reel avec callback asynchrone

```

import sounddevice as sd
import numpy as np
import queue

def capturer_avec_callback(
    duree_secondes: int = 10,
    taux_echantillonnage: int = 16000
) -> np.ndarray:
    """
    Capture audio en temps reel via callback asynchrone.
    Utile pour le traitement a la volee (detection
    d'evenements, VAD).
    """
    file_audio = queue.Queue()
    frames = []

    def callback(indata, frames_count, time_info, status):
        """Appelle pour chaque bloc audio capture."""
        if status:
            print(f"Statut : {status}")
            file_audio.put(indata.copy())

    print(f"Capture en cours ({duree_secondes}s)...")

    with sd.InputStream(
        samplerate=taux_echantillonnage,
        channels=1,
        callback=callback,
        blocksize=1024
    ):
        nb_blocs = int(duree_secondes * taux_echantillonnage /
                       1024)

```

```

    for _ in range(nb_blocs):
        bloc = file_audio.get()
        frames.append(bloc)

    audio = np.concatenate(frames, axis=0)
    print(f"Capture terminee : {len(audio)} echantillons")
    return audio

# Exemple
audio = capturer_avec_callback(duree_secondes=10)

```

2.4 Comparaison des Bibliotheques

Critere	PyAudio	SoundDevice	SpeechRecognition
Facilite d'usage	Moyenne	Elevee	Tres elevee
Performance	Elevee	Elevee	Moyenne
Formats	WAV (PCM)	WAV, FLAC	WAV, AIFF, FLAC
Callback async	Oui	Oui	Non
Dependances	PortAudio	PortAudio + NumPy	PyAudio
Acces numpy	Manuel	Natif	Non

2.5 Bonnes Pratiques

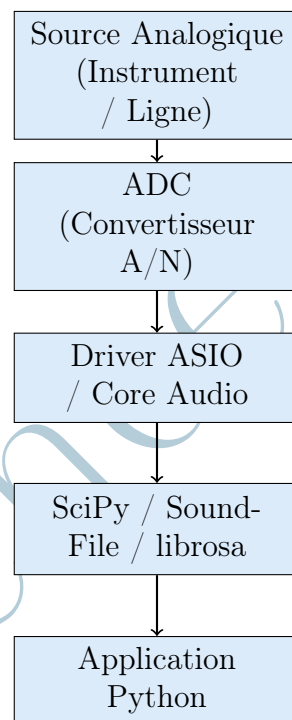
- **Calibration** : Effectuez une periode de calibration du bruit ambiant avant chaque session
- **Taux ASR** : Utilisez 16 kHz mono pour tous les modeles de reconnaissance vocale
- **Profondeur** : 16 bits pour la diffusion, 32 bits flottants pour le traitement numerique
- **Tampon** : Ajustez la taille du chunk selon la latence requise (512 = faible latence)
- **Permissions** : Sur macOS et Windows, autorisez l'accès au microphone dans les parametres

Chapitre 3 Carte Son et Fichiers Audio

3.1 Principe Fondamental

La carte son (ou interface audio) est le composant matériel qui réalise la conversion analogique-numérique (ADC) pour l'enregistrement et numérique-analogique (DAC) pour la lecture. En Python, les bibliothèques SciPy et SoundFile permettent de lire, écrire et manipuler des fichiers audio de tous formats.

3.1.1 Architecture Carte Son



3.1.2 Formats Audio Supportés

- **WAV** : Format non compressé, qualité parfaite, fichiers volumineux. Standard pour l'ASR.
- **FLAC** : Compression sans perte, 50% plus petit que WAV. Idéal pour l'archivage.
- **MP3 / OGG** : Compression avec perte, petits fichiers. Pour la distribution.
- **AIFF** : Équivalent Apple du WAV, non compressé, très haute qualité.
- **Opus** : Codec moderne basse latence, idéal pour le streaming temps réel.

3.2 Lecture avec SciPy

SciPy offre une interface simple et rapide pour les fichiers WAV.

Listing 3.1: Installation de SciPy

```
pip install scipy numpy
```

Listing 3.2: Lecture et analyse d'un fichier WAV avec SciPy

```
from scipy.io import wavfile
import numpy as np

def analyser_audio_scipy(chemin_fichier: str) -> dict:
    """
    Lit et analyse un fichier WAV avec SciPy.

    Returns:
        Dictionnaire avec metadonnees et statistiques du signal
    """
    # Lecture du fichier WAV
    taux, donnees = wavfile.read(chemin_fichier)

    # Conversion en float32 normalise
    if donnees.dtype == np.int16:
        donnees_float = donnees.astype(np.float32) / 32768.0
    elif donnees.dtype == np.int32:
        donnees_float = donnees.astype(np.float32) /
            2147483648.0
    else:
        donnees_float = donnees.astype(np.float32)

    # Statistiques du signal
    duree = len(donnees) / taux
    canaux = 1 if donnees.ndim == 1 else donnees.shape[1]
    amplitude_max = np.max(np.abs(donnees_float))
    rms = np.sqrt(np.mean(donnees_float**2))

    print(f"Fichier      : {chemin_fichier}")
    print(f"Taux          : {taux} Hz")
    print(f"Duree         : {duree:.3f}s")
    print(f"Canaux        : {canaux}")
    print(f"Echantillons  : {len(donnees)}")
    print(f"Amplitude max : {amplitude_max:.4f}")
    print(f"RMS           : {rms:.4f}")

    return {
        "taux_echantillonnage": taux,
        "duree": duree,
        "canaux": canaux,
        "nb_echantillons": len(donnees),
        "amplitude_max": float(amplitude_max),
        "rms": float(rms),
```

```

        "donnees": donnees_float
    }

# Exemple
info = analyser_audio_scipy("enregistrement.wav")

```

3.2.1 Ecriture d'un Fichier WAV

Listing 3.3: Generation et ecriture d'un signal audio

```

from scipy.io import wavfile
import numpy as np

def generer_tonalite(
    frequence: float = 440.0,
    duree: float = 1.0,
    taux: int = 44100,
    nom_fichier: str = "tonalite.wav"
) -> str:
    """
    Genere une tonalite sinusoidale et la sauvegarde en WAV.

    Args:
        frequence: Frequence en Hz (440 = La, 261 = Do)
        duree: Duree en secondes
    """
    t = np.linspace(0, duree, int(taux * duree), endpoint=False)
    signal = (np.sin(2 * np.pi * frequence * t) *
              32767).astype(np.int16)

    wavfile.write(nom_fichier, taux, signal)

    print(f"Tonalite {frequence} Hz ({duree}s) : {nom_fichier}")
    return nom_fichier

# Exemple : La concert a 440 Hz pendant 3 secondes
generer_tonalite(440.0, 3.0, nom_fichier="la_concert.wav")

```

3.3 Lecture avec SoundFile

SoundFile supporte davantage de formats : WAV, FLAC, OGG, AIFF et plus encore.

Listing 3.4: Installation de SoundFile

```
pip install soundfile
```

Listing 3.5: Lecture multi-formats avec SoundFile

```

import soundfile as sf
import numpy as np

```

```

def lire_audio_soundfile(chemin: str) -> tuple:
    """
    Lit un fichier audio dans n'importe quel format supporte.

    Returns:
        (donnees_numpy, taux_echantillonnage)
    """
    # Informations sans lire le contenu
    info = sf.info(chemin)
    print(f"Format          : {info.format}")
    print(f"Sous-type         : {info.subtype}")
    print(f"Taux              : {info.samplerate} Hz")
    print(f"Canaux            : {info.channels}")
    print(f"Duree             : {info.duration:.3f}s")
    print(f"Echantillons      : {info.frames}")

    # Lecture des donnees
    donnees, taux = sf.read(chemin, dtype='float32')

    return donnees, taux

def convertir_format(
    chemin_entree: str,
    chemin_sortie: str,
    format_sortie: str = "WAV"
) -> None:
    """
    Convertit un fichier audio d'un format a un autre.

    Args:
        format_sortie: 'WAV', 'FLAC', 'OGG', 'AIFF'
    """
    donnees, taux = sf.read(chemin_entree, dtype='float32')
    sf.write(chemin_sortie, donnees, taux, format=format_sortie)
    print(f"Converti : {chemin_entree} -> {chemin_sortie} ({format_sortie})")

# Exemples
audio, fs = lire_audio_soundfile("enregistrement.flac")
convertir_format("audio.wav", "audio.flac", "FLAC")

```

3.4 Reechantillonnage et Normalisation

Listing 3.6: Reechantillonnage et normalisation avec librosa

```

import librosa
import soundfile as sf
import numpy as np

def preparer_audio_pour_asr(

```

```

chemin_entree: str,
chemin_sortie: str,
taux_cible: int = 16000
) -> np.ndarray:
    """
    Prepare un fichier audio pour la reconnaissance vocale
    (ASR) :
    - Conversion mono si necessaire
    - Reechantillonnage au taux cible
    - Normalisation de l'amplitude au pic

    Returns:
        Array numpy normalise pret pour un modele ASR
    """
    # Chargement (librosa convertit automatiquement en float32
    # mono)
    audio, taux_source = librosa.load(chemin_entree, sr=None,
                                     mono=True)

    print(f"Source : {taux_source} Hz -> Cible : {taux_cible}
          Hz")

    # Reechantillonnage si necessaire
    if taux_source != taux_cible:
        audio = librosa.resample(audio, orig_sr=taux_source,
                                 target_sr=taux_cible)

    # Normalisation : amplitude max = 1.0
    amplitude_max = np.max(np.abs(audio))
    if amplitude_max > 0:
        audio = audio / amplitude_max

    # Sauvegarde
    sf.write(chemin_sortie, audio, taux_cible)

    print(f"Sauvegarde : {chemin_sortie}")
    print(f"Duree : {len(audio) / taux_cible:.3f}s")

    return audio

# Exemple : preparer pour Whisper
audio_prepare = preparer_audio_pour_asr(
    "conference.mp3",
    "conference_16k.wav",
    taux_cible=16000
)

```

3.4.1 Detection d'Activite Vocale (VAD)

Listing 3.7: Detection des segments de parole


```

import numpy as np

def detecter_segments_silences(
    audio: np.ndarray,
    taux: int = 16000,
    seuil_energie: float = 0.01,
    duree_min_silence: float = 0.3
) -> list:
    """
    Detecte les segments de parole par analyse d'energie.

    Args:
        seuil_energie: Seuil RMS pour distinguer silence/parole
        duree_min_silence: Duree minimale d'un silence en
            secondes

    Returns:
        Liste de (debut_sec, fin_sec) des segments de parole
    """
    taille_frame = int(taux * 0.02) # Frames de 20ms
    nb_frames = len(audio) // taille_frame

    # Calcul de l'energie par frame
    energies = []
    for i in range(nb_frames):
        frame = audio[i * taille_frame:(i + 1) * taille_frame]
        rms = np.sqrt(np.mean(frame**2))
        energies.append(rms)

    energies = np.array(energies)
    est_parole = energies > seuil_energie

    # Detection des transitions silence <-> parole
    segments = []
    en_parole = False
    debut = 0.0
    min_frames_silence = int(duree_min_silence / 0.02)
    compteur_silence = 0

    for i, parole in enumerate(est_parole):
        t = i * 0.02
        if parole and not en_parole:
            debut = t
            en_parole = True
            compteur_silence = 0
        elif not parole and en_parole:
            compteur_silence += 1
            if compteur_silence >= min_frames_silence:
                segments.append((debut, t))
                en_parole = False
                compteur_silence = 0

```

```
        elif parole:
            compteur_silence = 0

    if en_parole:
        segments.append((debut, nb_frames * 0.02))

    print(f"Segments de parole detectes : {len(segments)}")
    for debut, fin in segments:
        print(f"    [{debut:.2f}s - {fin:.2f}s]
              ({fin-debut:.2f}s)")

    return segments

# Exemple
import soundfile as sf
audio, fs = sf.read("discours.wav", dtype='float32')
if audio.ndim > 1:
    audio = audio[:, 0] # Prendre le premier canal
segments = detecter_segments_silences(audio, fs,
    seuil_energie=0.01)
```

3.5 Bonnes Pratiques

- **Format** : WAV pour la qualite maximale (sans perte), FLAC pour archiver sans perte
- **Taux ASR** : Toujours reechantillonner a 16 kHz mono avant tout modele ASR
- **Normalisation** : Normaliser a -23 LUFS (standard broadcast) ou au pic (-1 dBFS)
- **Compatibilite** : Verifiez le format supporte avant tout pipeline de traitement
- **Profondeur** : 16 bits pour la diffusion, 32 bits flottants pour le traitement interne

Chapitre 4 Telechargement Audio

4.1 Principe Fondamental

Le telechargement audio via HTTP/HTTPS ou APIs permet d'obtenir des donnees sonores depuis le web. C'est essentiel pour constituer des datasets d'apprentissage automatique, acceder a des archives audio publiques, ou recuperer des ressources sonores pour des projets IA.

4.2 Telechargement Simple avec Requests

Listing 4.1: Installation de Requests

```
pip install requests soundfile
```

Listing 4.2: Telechargement basique d'un fichier audio

```
import requests
import soundfile as sf
from io import BytesIO
import os

def telecharger_audio(url: str, nom_fichier: str = None) -> str:
    """
    Telecharge un fichier audio depuis une URL.

    Returns:
        Chemin du fichier sauvegarde
    """
    # Requete HTTP
    response = requests.get(url, timeout=30)
    response.raise_for_status()

    # Verification du type MIME
    content_type = response.headers.get('Content-Type', '')
    types_valides = ['audio/', 'application/octet-stream']
    if not any(t in content_type for t in types_valides):
        raise ValueError(f"L'URL ne pointe pas vers un audio : {content_type}")

    # Nom de fichier automatique si non fourni
    if nom_fichier is None:
        nom_fichier = url.split('/')[-1] or f"audio_{hash(url)} % 100000}.wav"

    # Sauvegarde
```

```

with open(nom_fichier, 'wb') as f:
    f.write(response.content)

# Informations du fichier
info = sf.info(nom_fichier)
print(f"Telecharge : {nom_fichier}")
print(f"  Format : {info.format}")
print(f"  Duree   : {info.duration:.2f}s")
print(f"  Taux    : {info.samplerate} Hz")
print(f"  Taille  : {len(response.content) / 1024:.2f} Ko")

return nom_fichier

# Exemple
url =
    "https://www2.cs.uic.edu/~i101/SoundFiles/BabyElephantWalk60.wav"
chemin = telecharger_audio(url, "exemple.wav")

```

4.2.1 Telechargement Multiple

Listing 4.3: Telechargement de plusieurs fichiers audio

```

import requests
import os

def telecharger_audios(urls: list, dossier_sortie: str =
    "./audio") -> list:
    """
    Telecharge plusieurs fichiers audio.

    Returns:
        Liste des chemins des fichiers crees
    """
    os.makedirs(dossier_sortie, exist_ok=True)
    fichiers = []

    for i, url in enumerate(urls):
        try:
            response = requests.get(url, timeout=30)
            response.raise_for_status()

            # Extraction du nom de fichier depuis l'URL
            nom = url.split('/')[-1] or f"audio_{i:04d}.wav"
            chemin = f"{dossier_sortie}/{nom}"

            with open(chemin, 'wb') as f:
                f.write(response.content)

            fichiers.append(chemin)
            taille = len(response.content) / 1024

```

```

        print(f"[{i+1}/{len(urls)}] {chemin} ({taille:.1f} Ko)")

    except Exception as e:
        print(f"[{i+1}/{len(urls)}] ERREUR : {e}")

    print(f"\nTelecharges : {len(fichiers)}/{len(urls)}")
    return fichiers

# Exemple
urls = [
    "https://example.com/son1.wav",
    "https://example.com/son2.wav",
    "https://example.com/son3.wav"
]
fichiers = telecharger_audios(urls, "./mes_sons")

```

4.3 Telechargement Asynchrone

Pour telecharger des centaines de fichiers audio rapidement, utilisez `aiohttp` :

Listing 4.4: Installation des dependances asynchrones

```
pip install aiohttp aiofiles
```

Listing 4.5: Telechargement asynchrone haute performance

```

import aiohttp
import asyncio
import aiofiles
import os

async def telecharger_audio_async(
    session: aiohttp.ClientSession,
    url: str,
    nom_fichier: str
) -> bool:
    """
    Telecharge un fichier audio de maniere asynchrone.
    """
    try:
        async with session.get(url,
                               timeout=aiohttp.ClientTimeout(total=30)) as response:
            if response.status != 200:
                return False

            contenu = await response.read()

            async with aiofiles.open(nom_fichier, 'wb') as f:
                await f.write(contenu)
    
```

```

        return True
    except Exception as e:
        print(f"Erreur {url}: {e}")
        return False

async def telecharger_audios_async(
    urls: list,
    dossier_sortie: str = "./audio"
) -> int:
    """
    Telecharge plusieurs fichiers audio en parallele.

    Returns:
        Nombre de fichiers telecharges avec succes
    """
    os.makedirs(dossier_sortie, exist_ok=True)

    async with aiohttp.ClientSession() as session:
        taches = []

        for i, url in enumerate(urls):
            nom = url.split('/')[-1] or f"audio_{i:04d}.wav"
            chemin = f"{dossier_sortie}/{nom}"
            tache = telecharger_audio_async(session, url,
                                             chemin)
            taches.append(tache)

        resultats = await asyncio.gather(*taches)

        succes = sum(resultats)
        print(f"Telecharges : {succes}/{len(urls)}")

        return succes

# Exemple : 50 fichiers en parallele
urls = [f"https://example.com/son_{i:04d}.wav" for i in
        range(50)]
asyncio.run(telecharger_audios_async(urls, "./dataset_audio"))

```

4.4 Datasets Audio Publics

4.4.1 HuggingFace Datasets

HuggingFace offre un acces unifie a des centaines de datasets audio publics.

Listing 4.6: Installation de Datasets HuggingFace

```
pip install datasets soundfile
```

Listing 4.7: Telechargement depuis HuggingFace Datasets

```
from datasets import load_dataset
import soundfile as sf
import os

def telecharger_dataset_hf(
    nom_dataset: str = "speech_commands",
    split: str = "train",
    nb_exemples: int = 100,
    dossier_sortie: str = "./dataset"
) -> list:
    """
    Telecharge des exemples audio depuis HuggingFace.

    Args:
        nom_dataset: Nom du dataset sur HuggingFace
        split: 'train', 'test', ou 'validation'
        nb_exemples: Nombre d'exemples a telecharger

    Returns:
        Liste des metadonnees telechargees
    """
    os.makedirs(dossier_sortie, exist_ok=True)

    print(f"Chargement de {nom_dataset} ({split})...")
    dataset = load_dataset(nom_dataset,
                           split=f"{split}[:{nb_exemples}]")

    metadonnees = []

    for i, exemple in enumerate(dataset):
        audio_data = exemple['audio']
        taux = audio_data['sampling_rate']
        signal = audio_data['array']
        label = exemple.get('label', i)

        # Sauvegarde
        nom_fichier = f"{dossier_sortie}/exemple_{i:04d}.wav"
        sf.write(nom_fichier, signal, taux)

        metadonnees.append({
            "fichier": nom_fichier,
            "label": label,
            "taux": taux,
            "duree": len(signal) / taux
        })

        if i % 10 == 0:
            print(f" [{i+1}/{nb_exemples}] {nom_fichier}")

    print(f"Telechargement termine : {len(metadonnees)} fichiers")
```

```

    return metadonnees

# Exemple : Common Voice (français)
meta = telecharger_dataset_hf(
    "mozilla-foundation/common_voice_11_0",
    split="train",
    nb_exemples=50,
    dossier_sortie="./common_voice"
)

```

4.4.2 Extraction depuis YouTube

Listing 4.8: Installation de yt-dlp

```

pip install yt-dlp

```

Listing 4.9: Extraction audio depuis YouTube avec yt-dlp

```

import yt_dlp
import os

def extraire_audio_youtube(
    url: str,
    dossier_sortie: str = "./youtube_audio",
    format_audio: str = "wav",
    taux_cible: int = 16000
) -> str:
    """
    Extrait la piste audio d'une video YouTube.

    Args:
        url: URL YouTube de la video
        format_audio: Format de sortie ('wav', 'mp3', 'flac')
        taux_cible: Taux d'echantillonnage en Hz

    Returns:
        Chemin du fichier audio extrait
    """
    os.makedirs(dossier_sortie, exist_ok=True)

    options = {
        'format': 'bestaudio/best',
        'postprocessors': [{
            'key': 'FFmpegExtractAudio',
            'preferredcodec': format_audio,
            'preferredquality': '192',
        }],
        'outtmpl': f'{dossier_sortie}/%(title)s.%(ext)s',
        'quiet': False
    }

    with yt_dlp.YoutubeDL(options) as ydl:

```



```
info = ydl.extract_info(url, download=True)
titre = info['title']
nom_fichier = f"{dossier_sortie}/{titre}.{format_audio}"

print(f"Extrait : {nom_fichier}")
return nom_fichier

# Exemple
chemin = extraire_audio_youtube(
    "https://www.youtube.com/watch?v=EXEMPLE",
    dossier_sortie="./audio_youtube",
    format_audio="wav"
)
```

4.5 Bonnes Pratiques

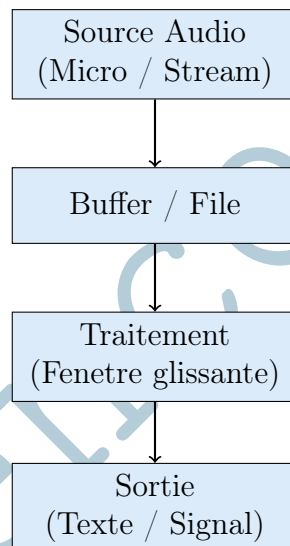
- **Respect** : Verifiez les CGU et les licences des datasets avant utilisation
- **Rate Limiting** : Ajoutez des delais entre les requetes (`time.sleep`)
- **Erreurs** : Gerez les timeouts, 404, et erreurs reseau avec `try/except`
- **Format** : Verifiez le Content-Type et l'integrite des fichiers audio
- **Deduplication** : Utilisez des hashes MD5 pour eviter les doublons

Chapitre 5 Streaming Audio en Temps Reel

5.1 Principe Fondamental

Le streaming audio permet de capturer et traiter des flux sonores continus sans attendre la fin de l'enregistrement. C'est la base de la transcription en temps réel, de la surveillance acoustique, et de la radio en ligne.

5.1.1 Architecture du Streaming



5.2 Streaming depuis le Microphone

Listing 5.1: Transcription en temps réel avec SoundDevice et Whisper

```
import sounddevice as sd
import numpy as np
import queue
import threading
import whisper

class TranscripteurTempsReel:
    """
    Transcription en temps réel depuis le microphone.
    Utilise une fenetre glissante avec Whisper.
    """

    def __init__(
```

```

self,
taux: int = 16000,
taille_fenetre: float = 3.0,
modele: str = "base"
):
    self.taux = taux
    self.taille_fenetre = taille_fenetre
    self.model = whisper.load_model(modele)
    self.file = queue.Queue()
    self.actif = False
    self.buffer = np.array([], dtype=np.float32)

def _callback(self, indata, frames, time, status):
    """Callback appele pour chaque bloc audio."""
    self.file.put(indata.copy())

def _traiter_buffer(self):
    """Thread de traitement du buffer audio."""
    while self.actif or not self.file.empty():
        try:
            bloc = self.file.get(timeout=0.5)
            self.buffer = np.append(self.buffer,
                                    bloc.flatten())

            # Traiter quand le buffer est plein
            nb_echantillons = int(self.taille_fenetre *
                                   self.taux)
            if len(self.buffer) >= nb_echantillons:
                fenetre = self.buffer[:nb_echantillons]
                resultat = self.model.transcribe(
                    fenetre,
                    language="fr",
                    fp16=False
                )
                texte = resultat["text"].strip()
                if texte:
                    print(f">> {texte}")
                # Fenetre glissante : garder la moitie
                self.buffer = self.buffer[nb_echantillons
                                           // 2:]

        except queue.Empty:
            continue

def demarrer(self):
    """Demarre la transcription en temps reel."""
    self.actif = True
    thread = threading.Thread(target=self._traiter_buffer)
    thread.start()

    print(f"Streaming actif (Ctrl+C pour arreter)...")

```

```

try:
    with sd.InputStream(
        samplerate=self.taux,
        channels=1,
        callback=self._callback,
        blocksize=int(self.taux * 0.5)
    ):
        while self.actif:
            sd.sleep(100)
except KeyboardInterrupt:
    self.actif = False
    thread.join()
    print("\nTranscription arretee.")

# Exemple
transcripteur = TranscriptionTempsReel(modele="base")
transcripteur.demarrer()

```

5.3 Streaming HTTP

Listing 5.2: Capture d'un flux audio HTTP (radio en ligne)

```

import requests
import numpy as np
import soundfile as sf
from io import BytesIO
import time

def capturer_stream_http(
    url_stream: str,
    duree_secondes: int = 30,
    nom_fichier: str = "stream_capture.wav"
) -> str:
    """
    Capture un flux audio HTTP (radio en ligne, podcast live).

    Args:
        url_stream: URL du flux audio
        duree_secondes: Duree de capture en secondes

    Returns:
        Chemin du fichier audio sauvegarde
    """
    print(f"Connexion au flux : {url_stream}")
    print(f"Capture pendant {duree_secondes}s...")

    contenu = BytesIO()
    debut = time.time()

```

```

with requests.get(url_stream, stream=True, timeout=10) as
    response:
        response.raise_for_status()

        for chunk in response.iter_content(chunk_size=8192):
            if time.time() - debut > duree_secondes:
                break
            if chunk:
                contenu.write(chunk)

# Sauvegarde des donnees brutes
contenu.seek(0)
with open(nom_fichier, 'wb') as f:
    f.write(contenu.read())

print(f"Capture sauvegardee : {nom_fichier}")
print(f"Taille : {contenu.tell() / 1024:.1f} Ko")

return nom_fichier

# Exemple : Radio en ligne (remplacez par une URL valide)
chemin = capturer_stream_http(
    "http://icecast.radiofrance.fr/franceinter-hifi.aac",
    duree_secondes=30
)

```

5.4 WebSocket pour l'Audio Temps Reel

Listing 5.3: Envoi d'audio en temps reel via WebSocket

```

import asyncio
import websockets
import sounddevice as sd
import numpy as np
import json

async def streamer_audio_websocket(
    uri: str,
    taux: int = 16000,
    duree: float = 10.0
):
    """
    Envoie l'audio du microphone vers un serveur WebSocket.
    Utile pour la transcription cloud en temps reel.
    """
    print(f"Connexion WebSocket : {uri}")

    async with websockets.connect(uri) as ws:
        # Envoyer la configuration
        config = {

```

```

        "type": "config",
        "sampleRate": taux,
        "encoding": "float32",
        "channels": 1
    }
    await ws.send(json.dumps(config))

    # Flux audio en temps reel
    nb_echantillons = int(taux * duree)
    audio = sd.rec(nb_echantillons, samplerate=taux,
                  channels=1, dtype='float32')

    BLOC = 1024 # Taille d'un bloc
    for i in range(0, nb_echantillons, BLOC):
        sd.wait() # Attendre que les donnees soient
                  disponibles
        bloc = audio[i:i+BLOC].flatten()
        # Envoyer le bloc en bytes
        await ws.send(bloc.tobytes())

    # Recevoir et afficher la transcription partielle
    try:
        reponse = await asyncio.wait_for(ws.recv(),
                                          timeout=0.1)
        data = json.loads(reponse)
        if data.get("type") == "partial":
            print(f"[Partiel] {data['text']}", end='\r')
        elif data.get("type") == "final":
            print(f"[Final] {data['text']}")
    except asyncio.TimeoutError:
        pass

    print("\nStreaming termine.")

# Exemple (remplacez l'URI par votre serveur WebSocket)
asyncio.run(streamer_audio_websocket("ws://localhost:8765"))

```

5.5 Bonnes Pratiques

- **Taille de bloc** : 512-1024 echantillons pour la faible latence, 4096+ pour l'efficacite
- **File d'attente** : Utilisez `queue.Queue` pour decoupler capture et traitement
- **Reconexion** : Implementez la reconnexion automatique pour les flux instables
- **Memoire** : Limitez la taille du buffer pour eviter les fuites memoire
- **Format** : Opus pour le streaming temps reel (faible latence, bonne compression)

Chapitre 6 Generation Audio par IA

6.1 Principe Fondamental

La generation audio par IA permet de creer synthetiquement des sons a partir de descriptions textuelles (text-to-speech) ou d'autres signaux (voice cloning, music generation). Les modeles comme Coqui TTS, Bark, et AudioCraft ont revolutionne ce domaine.

6.1.1 Modeles Principaux

Modele	Type	Acces	Qualite
Coqui TTS	Open-source	Local	Tres bonne
OpenAI TTS	Proprietaire	API	Excellente
ElevenLabs	Proprietaire	API	Excellente
Bark	Open-source	Local	Tres bonne
AudioCraft	Open-source	Local	Excellente

6.2 Synthese Vocale avec Coqui TTS

Coqui TTS est la reference open-source pour la synthese vocale en Python.

Listing 6.1: Installation de Coqui TTS

```
pip install TTS
```

Listing 6.2: Synthese vocale avec Coqui TTS

```
from TTS.api import TTS
import os

def generer_parole_coqui(
    texte: str,
    modele: str = "tts_models/fr/css10/vits",
    nom_fichier: str = "parole.wav",
    vitesse: float = 1.0
) -> str:
    """
    Synthetise de la parole depuis du texte avec Coqui TTS.

    Args:
        texte: Texte a synthetiser
        modele: ID du modele Coqui (voir tts --list_models)
        vitesse: Vitesse de la parole (1.0 = normale)

    Returns:
        Chemin du fichier audio genere
    """
```

```

"""
# Initialisation du modele
print(f"Chargement du modele '{modele}'...")
tts = TTS(model_name=modele, progress_bar=False)

# Synthese
print(f"Synthese : '{texte[:50]}...'")
tts.tts_to_file(
    text=texte,
    file_path=nom_fichier,
    speed=vitesse
)

print(f"Audio genere : {nom_fichier}")
return nom_fichier

# Exemple
texte = "Bonjour, je suis un systeme de synthese vocale en
Python."
chemin = generer_parole_coqui(texte, nom_fichier="bonjour.wav")

```

6.2.1 Parametres Avances

Listing 6.3: Synthèse avec voix personnalisée

```

from TTS.api import TTS

def synthetiser_avec_voix(
    texte: str,
    fichier_voix_reference: str,
    nom_sortie: str = "clone.wav",
    langue: str = "fr"
) -> str:
    """
    Synthetise la parole en clonant une voix de reference.

    Args:
        fichier_voix_reference: Fichier WAV de la voix a cloner
                               (5-30s)
        langue: Code de la langue cible
    """
    # Modele de clonage zero-shot
    tts =
        TTS(model_name="tts_models/multilingual/multi-dataset/xtts_v2")

    tts.tts_to_file(
        text=texte,
        speaker_wav=fichier_voix_reference,
        language=langue,
        file_path=nom_sortie
    )

```



```

    print(f"Voix clonee : {nom_sortie}")
    return nom_sortie

# Exemple : cloner une voix
synthetiser_avec_voix(
    "Ce texte est lu avec la voix clonee.",
    "voix_reference.wav",
    "sortie_clonee.wav",
    langue="fr"
)

```

6.3 TTS via OpenAI API

Listing 6.4: Installation du SDK OpenAI

```
pip install openai
```

Listing 6.5: Synthèse vocale avec OpenAI TTS

```

from openai import OpenAI
from pathlib import Path

def generer_parole_openai(
    texte: str,
    api_key: str,
    voix: str = "alloy",
    modele: str = "tts-1",
    format_sortie: str = "mp3",
    nom_fichier: str = "parole_openai.mp3"
) -> str:
    """
    Synthetise de la parole avec l'API OpenAI TTS.

    Args:
        voix: 'alloy', 'echo', 'fable', 'onyx', 'nova',
              'shimmer'
        modele: 'tts-1' (rapide) ou 'tts-1-hd' (haute qualite)
        format_sortie: 'mp3', 'opus', 'aac', 'flac'
    """
    client = OpenAI(api_key=api_key)

    print(f"Synthese OpenAI TTS (voix: {voix}, modele:
          {modele})...")
    response = client.audio.speech.create(
        model=modele,
        voice=voix,
        input=texte,
        response_format=format_sortie
    )

```

```

response.stream_to_file(nom_fichier)

taille = Path(nom_fichier).stat().st_size / 1024
print(f"Audio genere : {nom_fichier} ({taille:.1f} Ko)")
return nom_fichier

# Exemple
texte = "The quick brown fox jumps over the lazy dog."
generer_parole_openai(texte, api_key="VOTRE_CLE", voix="nova",
    modele="tts-1-hd")

```

6.4 Generation Musicale avec AudioCraft

AudioCraft (Meta) permet de generer de la musique et des sons a partir de descriptions textuelles.

Listing 6.6: Installation d'AudioCraft

```
pip install audiocraft
```

Listing 6.7: Generation musicale avec MusicGen

```

import torch
from audiocraft.models import MusicGen
from audiocraft.data.audio import audio_write
import soundfile as sf
import numpy as np

def generer_musique(
    description: str,
    duree: float = 8.0,
    modele: str = "small",
    nom_fichier: str = "musique"
) -> str:
    """
    Genere de la musique depuis une description textuelle.

    Args:
        description: Description de la musique a generer
        duree: Duree en secondes
        modele: 'small', 'medium', 'large', 'melody'

    Returns:
        Chemin du fichier audio genere (.wav)
    """
    print(f"Chargement de MusicGen ({modele})...")
    model = MusicGen.get_pretrained(modele)
    model.set_generation_params(duration=duree)

    print(f"Generation : '{description[:60]}...'")
    with torch.no_grad():

```

```
wav = model.generate([description])

# Sauvegarde
audio_write(
    nom_fichier,
    wav[0].cpu(),
    model.sample_rate,
    strategy="loudness"
)

print(f"Musique generee : {nom_fichier}.wav")
return f"{nom_fichier}.wav"

# Exemple
description = "An upbeat electronic track with synthesizers and drums"
chemin = generer_musique(description, duree=10.0,
    modele="small")
```

6.5 Bonnes Pratiques

- **GPU** : Un GPU avec 4+ Go de VRAM accelere considerablement la generation
- **Prompts** : Soyez precis dans vos descriptions textuelles pour TTS et generation
- **Clonage** : Pour le clonage vocal, utilisez 10-30s d'audio propre (pas de bruit)
- **Format** : WAV pour la qualite maximale, MP3/Opus pour la compression
- **Iteration** : Generez plusieurs variantes et selectionnez la meilleure

Chapitre 7 Extraction Audio depuis Video

7.1 Principe Fondamental

L'extraction audio consiste à isoler la piste sonore d'un fichier video. C'est essentiel pour la transcription automatique de conférences, le sous-titrage, la création de datasets audio, ou la séparation de sources musicales.

7.2 Extraction avec ffmpeg-python

Listing 7.1: Installation de ffmpeg-python

```
pip install ffmpeg-python
# Installer aussi le binaire ffmpeg : sudo apt-get install
ffmpeg
```

Listing 7.2: Extraction de la piste audio d'une video

```
import ffmpeg
import os

def extraire_audio_video(
    chemin_video: str,
    chemin_sortie: str = None,
    format_audio: str = "wav",
    taux_echantillonnage: int = 16000,
    canaux: int = 1
) -> str:
    """
    Extrait la piste audio d'un fichier video.

    Args:
        format_audio: Format de sortie ('wav', 'mp3', 'flac',
            'aac')
        taux_echantillonnage: Taux de sortie en Hz
        canaux: 1 = mono, 2 = stereo

    Returns:
        Chemin du fichier audio extrait
    """
    if chemin_sortie is None:
        base = os.path.splitext(chemin_video)[0]
        chemin_sortie = f"{base}.{format_audio}"

    print(f"Extraction : {chemin_video} -> {chemin_sortie}")
```

```

(
    ffmpeg
    .input(chemin_video)
    .output(
        chemin_sortie,
        acodec='pcm_s16le' if format_audio == 'wav' else
            format_audio,
        ac=canaux,
        ar=taux_echantillonnage
    )
    .overwrite_output()
    .run(quiet=True)
)

taille = os.path.getsize(chemin_sortie) / 1024
print(f"Audio extrait : {chemin_sortie} ({taille:.1f} Ko)")
return chemin_sortie

# Exemple
chemin_audio = extraire_audio_video(
    "conference.mp4",
    format_audio="wav",
    taux_echantillonnage=16000,
    canaux=1
)

```

7.2.1 Extraction par Intervalle Temporel

Listing 7.3: Extraction d'un segment audio

```

import ffmpeg

def extraire_segment_audio(
    chemin_video: str,
    debut: float,
    fin: float,
    chemin_sortie: str = "segment.wav"
) -> str:
    """
    Extrait un segment audio spécifique d'une video.

    Args:
        debut: Temps de debut en secondes
        fin: Temps de fin en secondes
    """
    duree = fin - debut

    (
        ffmpeg
        .input(chemin_video, ss=debut, t=duree)
    )

```

```

        .output(chemin_sortie, acodec='pcm_s16le', ac=1,
                ar=16000)
        .overwrite_output()
        .run(quiet=True)
    )

    print(f"Segment [{debut:.1f}s-{fin:.1f}s] ->
          {chemin_sortie}")
    return chemin_sortie

# Exemple : extraire entre 10s et 30s
chemin = extraire_segment_audio("video.mp4", debut=10.0,
                                fin=30.0)

```

7.3 Extraction avec pydub

Listing 7.4: Installation de pydub

```
pip install pydub
```

Listing 7.5: Manipulation audio avec pydub

```

from pydub import AudioSegment
import os

def decouper_et_normaliser(
    chemin_entree: str,
    dossier_sortie: str = "./segments",
    duree_segment_ms: int = 10000,
    normaliser: bool = True
) -> list:
    """
    Decoupe un fichier audio en segments de duree fixe et
    normalise.

    Args:
        duree_segment_ms: Duree de chaque segment en
                          millisecondes
        normaliser: Normaliser le volume de chaque segment

    Returns:
        Liste des chemins des segments crees
    """
    os.makedirs(dossier_sortie, exist_ok=True)

    print(f"Chargement : {chemin_entree}")
    audio = AudioSegment.from_file(chemin_entree)

    print(f"Duree totale : {len(audio) / 1000:.1f}s")
    print(f"Decoupage en segments de {duree_segment_ms /
        1000:.1f}s")

```

```

fichiers = []
nb_segments = len(audio) // duree_segment_ms

for i in range(nb_segments):
    debut = i * duree_segment_ms
    fin = debut + duree_segment_ms
    segment = audio[debut:fin]

    # Normalisation du volume
    if normaliser:
        changement_dB = -20.0 - segment.dBFS
        segment = segment.apply_gain(changement_dB)

    nom = f"{dossier_sortie}/segment_{i:04d}.wav"
    segment.export(nom, format="wav")
    fichiers.append(nom)

    if i % 10 == 0:
        print(f"  Segment {i+1}/{nb_segments}")

print(f"Segments crees : {len(fichiers)}")
return fichiers

# Exemple
segments = decouper_et_normaliser("conference.mp4",
    duree_segment_ms=10000)

```

7.4 Separation de Sources avec Demucs

Listing 7.6: Installation de Demucs

```
pip install demucs
```

Listing 7.7: Separation vocale/instrumentale avec Demucs

```

import subprocess
import os

def separer_sources_audio(
    chemin_entree: str,
    dossier_sortie: str = "./separation",
    modele: str = "htdemucs"
) -> dict:
    """
    Separe les sources audio (voix, basse, batterie, autre).

    Args:
        modele: 'htdemucs', 'mdx_extra', 'demucs'

    Returns:
    """

```

```

    Dictionnaire {source: chemin_fichier}
"""
os.makedirs(dossier_sortie, exist_ok=True)

print(f"Separation avec {modele}...")
cmd = [
    "python", "-m", "demucs",
    "--name", modele,
    "--out", dossier_sortie,
    chemin_entree
]

result = subprocess.run(cmd, capture_output=True, text=True)

if result.returncode != 0:
    raise RuntimeError(f"Demucs error: {result.stderr}")

# Les fichiers sont dans dossier_sortie/modele/nom_fichier/
nom_base =
    os.path.splitext(os.path.basename(chemin_entree))[0]
dossier_resultat = os.path.join(dossier_sortie, modele,
    nom_base)

sources = {}
for source in ["vocals", "drums", "bass", "other"]:
    chemin = os.path.join(dossier_resultat, f"{source}.wav")
    if os.path.exists(chemin):
        sources[source] = chemin
        print(f" {source}: {chemin}")

return sources

# Exemple
sources = separer_sources_audio("chanson.mp3",
    modele="htdemucs")
print(f"Voix isolee : {sources.get('vocals', 'N/A')}")

```

7.5 Transcription Automatique avec Whisper

Listing 7.8: Pipeline extraction + transcription complet

```

import ffmpeg
import whisper
import os
import json

def transcrire_video(
    chemin_video: str,
    modele: str = "small",
    langue: str = None,

```



```

dossier_sortie: str = "."
) -> dict:
    """
    Pipeline complet : extraction audio + transcription Whisper.

    Args:
        modele: Taille du modele Whisper (tiny, base, small,
            medium, large)
        langue: Langue forcee (None = detection automatique)

    Returns:
        Dictionnaire avec texte complet et segments horodates
    """
    # Etape 1 : Extraction audio
    base = os.path.splitext(os.path.basename(chemin_video))[0]
    chemin_audio = f"{dossier_sortie}/{base}_audio.wav"

    print(f"Extraction audio...")
    (
        ffmpeg.input(chemin_video)
        .output(chemin_audio, acodec='pcm_s16le', ac=1,
            ar=16000)
        .overwrite_output()
        .run(quiet=True)
    )

    # Etape 2 : Transcription
    print(f"Transcription avec Whisper ({modele})...")
    model = whisper.load_model(modele)
    options = {"language": langue} if langue else {}
    resultat = model.transcribe(chemin_audio, **options)

    # Sauvegarde de la transcription
    chemin_json = f"{dossier_sortie}/{base}_transcription.json"
    with open(chemin_json, 'w', encoding='utf-8') as f:
        json.dump({
            "fichier": chemin_video,
            "langue": resultat["language"],
            "texte": resultat["text"],
            "segments": [
                {
                    "debut": seg["start"],
                    "fin": seg["end"],
                    "texte": seg["text"]
                }
                for seg in resultat["segments"]
            ]
        }, f, ensure_ascii=False, indent=2)

    print(f"Transcription sauvegardee : {chemin_json}")
    os.remove(chemin_audio) # Nettoyage

```

```
    return resultat

# Exemple
resultat = transcrire_video("conference.mp4", modele="small",
                             langue="fr")
print(f"\nTexte complet :\n{resultat['text']}")
```

7.6 Bonnes Pratiques

- **Format** : WAV 16 kHz mono pour l'ASR, FLAC pour l'archivage sans perte
- **Espace disque** : 1h de video $\approx$ 115 Mo en WAV 16 kHz, surveillez l'espace
- **Traitement par lots** : Utilisez le batch processing pour les grands volumes
- **Reechantillonnage** : Toujours reechantillonner a 16 kHz mono avant l'ASR
- **Codec** : Utilisez pcm_s16le pour un WAV compatible avec tous les outils

Chapitre 8 Conclusion

8.1 Tableau Comparatif des Mecanismes

Mecanisme	Use Cases	Outils Python	Qualite
Microphone	Dictee, ASR, surveillance	PyAudio, SoundDevice	Variable
Carte Son / ADC	Instruments, studio	SciPy, SoundFile	Excellente
Telechargement	Datasets ML, archives	requests, aiohttp	Variable
Streaming	Radio, transcription live	SoundDevice, Web-Socket	Temps reel
Generation IA	TTS, musique, clonage	Coqui TTS, AudioCraft	Synthetique
Extraction Video	Sous-titrage, analyse	ffmpeg, pydub, demucs	Variable

8.2 Bonnes Pratiques Generales

1. Choisir le bon mecanisme selon la source

- Voix ambient ? → Microphone (PyAudio / SoundDevice)
- Instrument / Mixage ? → Carte Son (SciPy / SoundFile)
- Web / Dataset ? → Telechargement (requests / aiohttp)
- Flux continu ? → Streaming (WebSocket / HTTP)
- Synthetique ? → Generation IA (Coqui TTS / AudioCraft)
- Video ? → Extraction (ffmpeg / pydub / demucs)

2. Valider la qualite

- Verifiez le taux d'echantillonnage, le format, et la duree
- Mesurez le SNR (Signal-to-Noise Ratio) pour les enregistrements
- Controlez la taille des fichiers et la corruption eventuelle

3. Gerer les erreurs

- Try/except pour toutes les operations I/O audio
- Timeouts pour les telechargements et les flux reseau
- Verifiez `stream.is_active()` avant la lecture

4. Optimiser les performances

- Asynchrone pour les telechargements multiples (aiohttp)

- GPU pour la generation IA et la transcription Whisper
- Batch processing pour l'extraction de grandes collections video

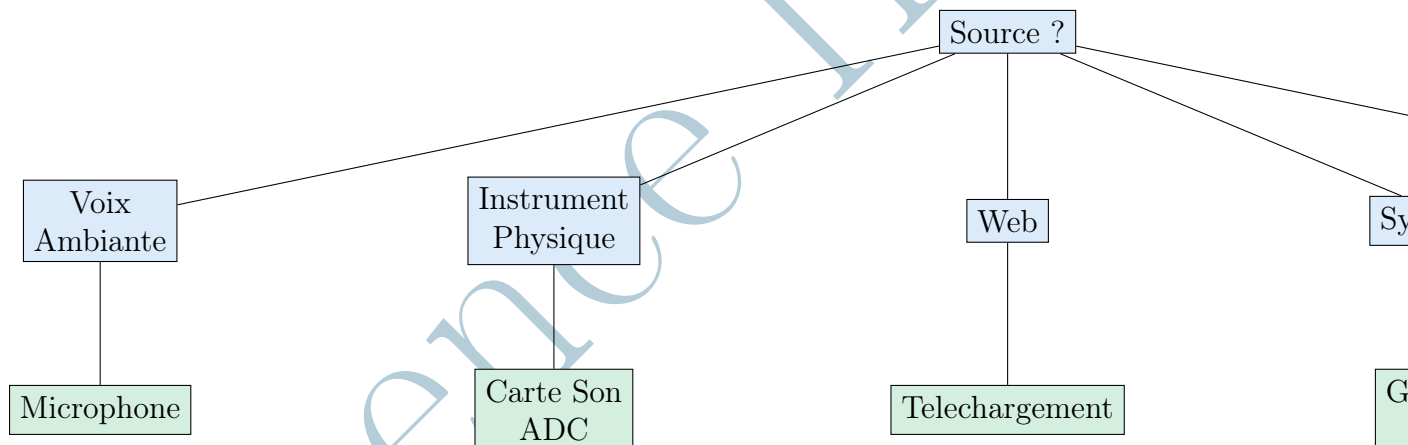
5. Respecter les standards audio

- WAV PCM pour la qualite sans perte
- FLAC pour la compression sans perte (archivage)
- Opus pour le streaming temps reel (faible latence)
- MP3 / AAC pour la distribution (compression avec perte acceptable)

6. Documenter et versionner

- Timestamps dans les noms de fichiers
- Metadata (taux, canaux, format, source, date)
- Logs d'acquisition et registres d'erreurs

8.3 Arbre de Decision



Conseil

Pour un projet IA audio, combinez plusieurs mecanismes :

- Dataset initial : Telechargement de datasets publics (LibriSpeech, Common Voice)
- Augmentation : Generation IA pour diversifier les donnees
- Production : Microphone pour l'inference en temps reel

Chapitre 9 Bibliographie et Ressources

9.1 Ouvrages de Reference

Pour approfondir vos connaissances, voici les ouvrages recommandes :

- Muller, M. (2015). *Fundamentals of Music Processing : Audio, Analysis, Algorithms, Applications*. Springer. — Reference complete en traitement du signal audio et musical.
- Geron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. O'Reilly Media. — Excellent pour comprendre les modeles de reconnaissance vocale et de generation audio.
- Radford, A. et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision*. OpenAI. — Article fondateur decrivant le modele Whisper.
- Copet, J. et al. (2023). *Simple and Controllable Music Generation*. NeurIPS. — Article fondateur de MusicGen / AudioCraft (Meta).
- Casanova, E. et al. (2022). *XTTS: A Massively Multilingual Zero-Shot Text-to-Speech Model*. Interspeech. — Base du modele de clonage vocal XTTS.
- Panayotov, V. et al. (2015). *LibriSpeech: An ASR Corpus Based on Public Domain Audio Books*. ICASSP. — Dataset de reference pour la reconnaissance vocale.

9.2 Documentation Officielle

Outil	URL	Description
PyAudio	people.csail.mit.edu/hubert/pyaudio	Capture microphone
SoundDevice	python-sounddevice.readthedocs.io	Capture / Lecture NumPy
SoundFile	pysoundfile.readthedocs.io	Lecture / Ecriture audio
Whisper	github.com/openai/whisper	STT offline OpenAI
Coqui TTS	tts.readthedocs.io	Synthese vocale open source
AudioCraft	github.com/facebookresearch/audiocraft	Generation musicale Meta
ffmpeg-python	github.com/kkroening/ffmpeg-python	Extraction / Conversion
pydub	github.com/jiaaro/pydub	Manipulation audio haut niveau

9.3 Tutoriels Recommandes

- **Real Python** (realpython.com) — Tutoriels de qualite professionnelle :
 - Speech Recognition in Python
 - Working With Audio Files in Python

– Playing and Recording Sound in Python

- **Hugging Face** (huggingface.co/docs) — Documentation complete sur les modèles audio (Whisper, Wav2Vec 2.0, SpeechT5).
- **PyImageSearch Audio** (pyimagesearch.com) — Guides pour la vision par ordinateur et le traitement du signal.

9.4 Jeux de Données pour Pratiquer

- **LibriSpeech** (www.openslr.org/12) — 960 heures d’anglais parlé, référence pour l’ASR.
- **Common Voice** (voice.mozilla.org) — Base de données audio multilingue, dont le français (100+ langues).
- **UrbanSound8K** (urbansounddataset.weebly.com) — 8 732 extraits de sons urbains classés en 10 catégories.
- **FreeSound** (freesound.org) — Plus de 500 000 sons libres de droits, avec API de téléchargement.
- **VCTK** (datashare.ed.ac.uk/handle/10283/2651) — 110 locuteurs anglais pour le clonage vocal et la synthèse.
- **MagnaTagATune** (mirg.city.ac.uk/codetools/magnatagatune) — 25 863 extraits musicaux annotés, pour la classification musicale.

9.5 Aspects Legaux et Ethiques

- **Consentement** — L’enregistrement audio de personnes sans leur consentement est illégal dans la plupart des pays (RGPD en Europe, wiretapping laws aux USA).
- **Données biométriques** — La voix est considérée comme une donnée biométrique par le RGPD : traitement soumis à des obligations spécifiques.
- **Droits d’auteur** — Vérifiez les licences avant d’utiliser des audios téléchargés. Creative Commons, domaine public, ou licences commerciales.
- **Deepfakes audio** — Le clonage vocal malveillant (usurpation d’identité, fraude) est illégal dans de nombreux pays.
- **Biais des modèles** — Les modèles ASR peuvent avoir des performances différentes selon les accents, les langues ou les conditions d’enregistrement. Testez sur des données diversifiées.

Conseil

Pour un démarrage rapide :

- Enregistrement microphone : **sounddevice** (simple, NumPy natif)
- Lecture / Conversion fichiers : **soundfile** (multi-formats)
- Transcription : Whisper (**small**) – gratuit, offline, 99+ langues

- Generation vocale : Coqui TTS – gratuit, open source, local
- Extraction video : `ffmpeg-python` – rapide et universel

Licence IRM